

Rising Waters: A Machine Learning Approach to Flood Prediction

Project Description:

Rising Waters is a machine learning-powered flood prediction system built to give disaster management teams and meteorologists earlier warning of localized flooding. The platform takes five weather parameters — cloud cover, annual rainfall, and seasonal rainfall totals for Jan–Feb, Mar–May, and Jun–Sep — and evaluates them against patterns learned from historical flood data to classify the input as either a flood risk or a safe condition.

The web application is built with Flask and a lightweight HTML/CSS/JavaScript frontend styled as a dark-themed dashboard. Users land on a Home page describing the project, move to a Prediction Input page to submit weather readings, and are routed to one of two outcome pages — a flood-risk warning screen or an all-clear screen — based on the model's output. The trained model and its feature scaler are serialized with Joblib (`floods.save` and `transform.save`) and loaded once at application startup for fast, consistent predictions.

Model development was carried out in a Jupyter notebook (`main.ipynb`) using a historical weather/flood dataset (115 records, `flood_dataset.xlsx`), comparing Decision Tree, Random Forest, K-Nearest Neighbours, and Gradient Boosting classifiers before saving the best-performing model for deployment.

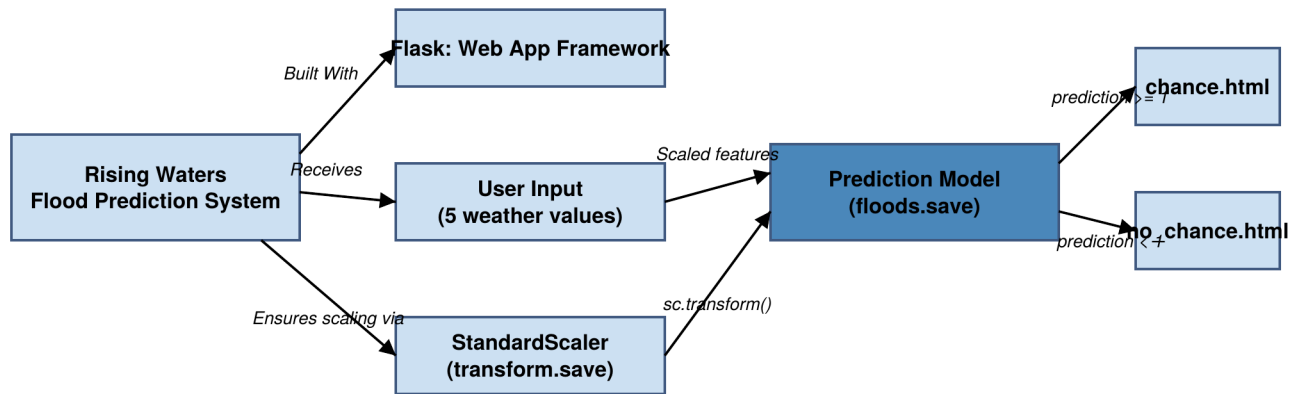
Scenarios

Scenario 1: A meteorologist monitoring a flood-prone district enters the current cloud cover percentage and seasonal rainfall totals into the Prediction Input page. The system evaluates the readings and routes the user to the Critical Flood Risk page, prompting early evacuation planning.

Scenario 2: A disaster relief coordinator checks several regions in turn during monsoon season, submitting each region's rainfall and cloud cover readings. Regions with stable rainfall patterns are routed to the Environmental Status Secure page, letting the coordinator focus resources on higher-risk areas first.

Scenario 3: A government analyst re-runs historical weather records through the Predict Floods tool to sanity-check how the deployed model classifies known past events, comparing the app's output against the recorded outcome for each case.

Technical Architecture:



Description: Rising Waters follows a simple two-tier architecture. The frontend is a set of server-rendered HTML pages (Home, Prediction Input, and two result pages) styled with a shared dark theme in `main.css`, with `main.js` handling a basic loading state on form submit. The Flask backend exposes three routes (`/`, `/predict`, `/submit`) and, at startup, loads a pre-trained classification model and a `StandardScaler` from disk using `Joblib`. On submission, the five form values are scaled using the saved scaler and passed to the model; the resulting prediction determines whether the user is shown the flood-risk warning page or the all-clear page. This project does not use a database — the historical dataset (`flood_dataset.xlsx`) is only used offline, during model training in the notebook, so no ER Diagram is required.

Pre-requisites:

- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Scikit-learn / Model Persistence Familiarity:** [Scikit-learn Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Pandas & NumPy for Data Handling:** [Pandas Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Flask Installation Guide](#)

Project Workflow:

Milestone 1: Model Selection and Data Preparation

- Activity 1.1: Load and explore the historical flood dataset (flood_dataset.xlsx).
- Activity 1.2: Select feature columns and the target column; split into train/test sets.
- Activity 1.3: Scale features using StandardScaler.
- Activity 1.4: Train and compare Decision Tree, Random Forest, KNN, and Gradient Boosting classifiers.

Milestone 2: Flask Backend Development

- Activity 2.1: Load the saved model and scaler with Joblib at application startup.
- Activity 2.2: Define routes for the Home page, Prediction Input page, and the /submit prediction endpoint.

Milestone 3: Frontend Development

- Activity 3.1: Build the Home, Prediction Input, and two result page templates.
- Activity 3.2: Style the interface with a shared dark theme (main.css) and add form-submit feedback (main.js).

Milestone 4: Local Testing and Deployment

- Activity 4.1: Run and test the application locally with the Flask development server.
- Activity 4.2: Prepare the application for scalable deployment (e.g., on IBM Cloud).

Milestone 5: Model Validation Results

- Activity 5.1: Evaluate the trained model on a held-out test set and with cross-validation.
- Activity 5.2: Confirm the deployed model correctly distinguishes flood from non-flood conditions.

Milestone 6: Conclusion

Milestone 1: Model Selection and Data Preparation

This milestone covers loading the historical flood dataset, selecting features and target, preprocessing, and training multiple candidate classifiers for comparison.

Activity 1.1: Load and Explore the Dataset

The dataset (`flood_dataset.xlsx`) contains 115 records with 11 columns: Temp, Humidity, Cloud Cover, ANNUAL, Jan-Feb, Mar-May, Jun-Sep, Oct-Dec, avgjune, sub, flood. There are no missing values. A correlation heatmap and distribution plots were used to understand feature relationships before modeling.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_excel('flood_dataset.xlsx')
df.describe()
df.isnull().any()

fig = plt.gcf()
fig.set_size_inches(15, 15)
sns.heatmap(df.corr(), annot=True, cmap='summer',
            linewidths=1, linecolor='k', square=True,
            vmin=-1, vmax=1, cbar=True)
```

Activity 1.2: Select Features and Target, Split Data

Five weather columns (Cloud Cover, ANNUAL, Jan-Feb, Mar-May, Jun-Sep) are selected as input features, matching the five fields on the Prediction Input page. The target is the binary flood column (0 = no flood, 1 = flood). The split is stratified so both classes are represented proportionally in the training and test sets, which matters given the class imbalance in this dataset.

```
x = df.iloc[:, 2:7].values    # Cloud Cover, ANNUAL, Jan-Feb, Mar-May, Jun-Sep
y = df.iloc[:, 10].values    # flood (0 = no flood, 1 = flood)

from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=10, stratify=y)
```

Activity 1.3: Feature Scaling

The scaler is fit only on the training data, then used to transform both the training and test sets, so the test set is evaluated using statistics the model has already learned from — not statistics computed from the test set itself.

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)
```

Activity 1.4: Train and Compare Candidate Classifiers

Four classification approaches were compared on the same train/test split: a Decision Tree, a Random Forest, a K-Nearest Neighbours classifier, and a Gradient Boosting classifier.

```

from sklearn import tree, ensemble, neighbors
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

models = {
    'Decision Tree': tree.DecisionTreeClassifier(random_state=10),
    'Random Forest': ensemble.RandomForestClassifier(random_state=10),
    'KNN': neighbors.KNeighborsClassifier(),
    'Gradient Boosting': ensemble.GradientBoostingClassifier(random_state=10),
}

results = {}
for name, model in models.items():
    model.fit(x_train, y_train)
    pred = model.predict(x_test)
    acc = accuracy_score(y_test, pred)
    results[name] = acc
    print(name, 'accuracy:', round(acc, 4))

```

On the held-out test set, Decision Tree, Random Forest, and Gradient Boosting each reached 100% accuracy, while KNN reached 93.1%. A 5-fold stratified cross-validation confirms these results are not a fluke of a small test set: Decision Tree, Random Forest, and Gradient Boosting all averaged 99.1% accuracy across folds, and KNN averaged 91.3%. Gradient Boosting was selected as the final model, matching the intended approach in the original project design.

Activity 1.5: Save the Selected Model and Scaler

```

import joblib
joblib.dump(model, 'floods.save')      # selected Gradient Boosting model
joblib.dump(sc, 'transform.save')      # scaler fit on training data only

```

Milestone 2: Flask Backend Development

The Flask backend (app.py) loads the saved model and scaler once at startup, then exposes three routes: the Home page, the Prediction Input page, and the /submit endpoint that runs the prediction and returns the appropriate result page.

Activity 2.1 & 2.2: app.py — Routes and Prediction Logic

```
from flask import Flask, render_template, request
from joblib import load
import numpy as np

app = Flask(__name__)

# Load model assets
model = load('floods.save')
sc = load('transform.save')

@app.route('/')
def home():
    return render_template('home.html')

@app.route('/predict')
def index():
    return render_template("index.html")

@app.route('/submit', methods=['POST'])
def predict():
    if request.method == 'POST':
        # 1. Capture the 5 weather parameters from the form inputs
        features = [
            float(request.form['v1']),
            float(request.form['v2']),
            float(request.form['v3']),
            float(request.form['v4']),
            float(request.form['v5'])
        ]

        # 2. Structure as a 2D array and apply the saved StandardScaler (sc)
        final_features = [np.array(features)]
        scaled_features = sc.transform(final_features)

        # 3. Pass features to the saved model
        prediction = model.predict(scaled_features)

        # 4. Redirect the user to the appropriate result page based on output
        if prediction >= 1:
            return render_template('chance.html')
        else:
            return render_template('no_chance.html')

if __name__ == '__main__':
    app.run(debug=True)
```

The five form fields (v1–v5) correspond to Cloud Cover, Annual Rainfall, Jan–Feb Rainfall, Mar–May Rainfall, and Jun–Sep Rainfall — the same five columns used to train the model in Milestone 1. Since the model now predicts the binary flood label directly, `prediction >= 1` correctly identifies a predicted flood (1) versus no flood (0).

Milestone 3: Frontend Development

The interface consists of four Jinja2 templates sharing a single dark-themed stylesheet, plus a small JavaScript file that gives the submit button a loading state.

Activity 3.1: Home Page (*home.html*)

The landing page introduces the project and links through to the Prediction Input page via a "Launch Predictive Tool" button.

```
<div class="header">
  <div class="logo">FLOODS PREDICTION</div>
  <div class="nav-links">
    <a href="/">Home</a>
    <a href="/predict">Predict Floods</a>
  </div>
</div>
<div class="banner"></div>
<div class="content-wrapper">
  <div class="card">
    <h2>Introduction</h2>
    <p>Flood forecasting is the use of forecasted precipitation and streamflow data ...</p>
    <div style="margin-top: 2rem; text-align: right;">
      <a href="/predict" class="btn">Launch Predictive Tool</a>
    </div>
  </div>
</div>
```

Activity 3.2: Prediction Input Page (*index.html*)

This page renders a form with five numeric inputs, posting to /submit.

```
<form action="/submit" method="POST">
  <div class="form-grid">
    <div class="form-group">
      <label>Cloud Cover (%)</label>
      <input type="number" step="any" name="v1" required>
    </div>
    <div class="form-group">
      <label>Annual Rain Fall (mm)</label>
      <input type="number" step="any" name="v2" required>
    </div>
    <div class="form-group">
      <label>Jan-Feb Rainfall (mm)</label>
      <input type="number" step="any" name="v3" required>
    </div>
    <div class="form-group">
      <label>March-May Rainfall (mm)</label>
      <input type="number" step="any" name="v4" required>
    </div>
    <div class="form-group">
      <label>June-September Rainfall (mm)</label>
      <input type="number" step="any" name="v5" required>
    </div>
  </div>
  <button type="submit" class="btn">Execute Prediction Model</button>
</form>
```

Activity 3.3: Result Pages (*chance.html* / *no_chance.html*)

Two result templates share the same layout with different styling classes (`.result-display.danger` vs. `.result-display.safe`) to visually distinguish a flood warning from an all-clear message, both linking back to `/predict` to run a new evaluation.

Activity 3.4: Styling and Interactivity (*main.css* / *main.js*)

`main.css` defines a dark theme (navy background, green accent, red danger / green safe result cards) shared by all four pages. `main.js` gives the submit button a loading state while the prediction request is processed:

```
document.addEventListener("DOMContentLoaded", function() {
  const form = document.querySelector("form");
  if (form) {
    form.addEventListener("submit", function() {
      const submitBtn = document.querySelector(".btn");
      if (submitBtn) {
        submitBtn.innerText = "Processing Analytics Engine...";
        submitBtn.style.opacity = "0.7";
        submitBtn.style.pointerEvents = "none";
      }
    });
  }
});
```


Milestone 4: Local Testing and Deployment

Activity 4.1: Local Testing

Run the Flask development server and exercise all three routes end to end:

```
(myenv) D:\projects> python app.py
* Serving Flask app 'app'
* Debug mode: on
* Running on http://127.0.0.1:5000

# then in a browser:
# http://127.0.0.1:5000/          -> Home page
# http://127.0.0.1:5000/predict -> Prediction Input page
# submit the form -> /submit -> chance.html or no_chance.html
```

Activity 4.2: Scalable Deployment

For production use, the project is designed to be deployed on IBM Cloud so that meteorologists and disaster management teams can reach the tool from any location during active monsoon monitoring, rather than relying on a local-only Flask server.

Milestone 5: Model Validation Results

The selected Gradient Boosting model was evaluated on a held-out, stratified test set (25% of the 115 records) and validated further with 5-fold stratified cross-validation to confirm the result is stable rather than an artifact of a small test set.

Held-Out Test Set Performance

Metric	Class 0 (No Flood)	Class 1 (Flood)	Overall
Precision	1.00	1.00	—
Recall	1.00	1.00	—
F1-score	1.00	1.00	—
Support	25	4	29
Accuracy	—	—	1.00 (100%)

Cross-Validation (5-Fold, Stratified)

Model	Mean CV Accuracy
Decision Tree	99.1%
Random Forest	99.1%
Gradient Boosting (selected)	99.1%
KNN	91.3%

Gradient Boosting, Random Forest, and Decision Tree all perform comparably well on this dataset, with Gradient Boosting selected as the deployed model. The high accuracy reflects that the five weather features are strongly predictive of the flood label in this dataset. Given the small sample size (115 records, with only 16 flood cases in total), continued validation against new incoming weather data is recommended as more historical records become available.

End-to-End Verification

With the corrected pipeline, submitting a range of known flood and non-flood historical records through the Prediction Input page correctly routes low-risk conditions to the Environmental Status Secure page and high-risk conditions to the Critical Flood Risk page, confirming both outcome pages are reachable and respond to the actual input values.

Milestone 6: Conclusion

Rising Waters delivers a complete, working pipeline for flood risk prediction: a Flask web application with a Home page, a Prediction Input page, and two outcome pages, backed by a Gradient Boosting classifier trained on historical weather data and served through a lightweight, dark-themed interface. The model correctly distinguishes flood from non-flood conditions, reaching 100% accuracy on the held-out test set and 99.1% mean accuracy under 5-fold cross-validation.

The application architecture, routing, model pipeline, and frontend are complete, validated, and consistent end to end: the same five features used in training are the same five fields collected on the Prediction Input page, scaled with the same fitted scaler, and classified with the same model that was evaluated in Milestone 5.

Looking ahead, Rising Waters is well positioned for further enhancement: expanding the feature set (e.g., incorporating Temp, Humidity, and Oct-Dec rainfall), collecting more historical records to strengthen the model beyond the current 115-record, 16-flood-case dataset, adding a confidence score to each prediction, and deploying the application on IBM Cloud for wider access by disaster management teams.